AMITY UNIVERSITY NOIDA, UTTAR PRADESH

AMITY SCHOOL OF ENGINEERING AND TECHNOLOGY

OPERATING SYSTEM (CSE202)
LAB FILE

Submitted To:
Mr. Nirbhay Kashyap

Submitted By:
Manan
A2345924076
B.Tech. 4CSE Evng- 2X

Q1. Write Unix command for following.

I Write a command that shows the contents of a file called report.txt one page at a time.

more report.txt

II Write a command to show the current working directory and all files, even hidden ones.

pwd && ls -a

III Write a command to make a folder called "project" inside "/home/user/".

mkdir /home/user/project

IV Write a command that will tell you how many lines, words, and characters are in a file called data.log.

wc data.log

V Make a command that copies all .txt files from the current directory to /home/backup/.

cp *.txt /home/backup/

VI Write a command that will look for all files in the /var/log directory that are bigger than 10MB.

find /var/log -type f -size +10M

VII Write a command that will sort the file names.txt in reverse order.

sort -r names.txt

Q2. Write Unix command for following.

I Write a command that adds the output of the date command to a file called system.log.

date >> system.log

II Write a command that sends error messages from a command to a file called error.txt.

command 2> error.txt

III Write a pipeline that looks for the word "disk" in /var/log/syslog, counts how many times it appears, and only shows the count.

```
grep -o "disk" /var/log/syslog | wc -l
```

IV Write a command that combines the contents of f1.txt and f2.txt and saves the result in final.txt.

```
cat f1.txt f2.txt > final.txt
```

V Write a command that uses pipes to show the 10 most used commands from .bash history.

```
history | awk '{print $2}' | sort | uniq -c | sort -nr | head
```

VI **Write a command that compares the two files old.txt and new.txt and shows the differences**.

```
diff old.txt new.txt
```

VII Write a command that sorts, removes duplicates, and saves the results to a new file called unique.txt.

```
sort file.txt | uniq > unique.txt
```

Q3. Write Unix command for following.

I List all files (including hidden files) in the current directory, sorted by file size in descending order.

```
ls -laS
```

II Display the total number of lines, words, and characters in a file named report.txt

```
wc report.txt
```

III Find all .txt files in the /home/user/docs directory and its subdirectories that were modified in the last 7 days.

```
find /home/user/docs -name "*.txt" -mtime -7
```

IV Extract and display only the second and fourth columns from a space-separated file named data.txt.

```
awk '{print $2, $4}' data.txt
```

V Search for the word error (case-insensitive) in a log file system.log and redirect the output to errors.txt.

```
grep -i "error" system.log > errors.txt
```

VI Show the top 5 processes consuming the highest CPU usage on the system.

ps -eo pid,ppid,cmd,%cpu --sort=-%cpu | head -6

VII Compress the directory project/ into a file named project_backup.tar.gz.

tar -czvf project_backup.tar.gz project/

Q4. Write a C program to implement Non-Preemptive Shortest Job First (SJF) scheduling algorithm.

```
#include <stdio.h>
int main() {
    int n, bt[20], wt[20], tat[20], p[20];
    int i, j, temp;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for(i=0;i<n;i++){
        printf("Burst time P%d: ", i+1);
        scanf("%d", &bt[i]);
        p[i]=i+1;
    }
    // sort by burst time
    for(i=0;i<n;i++){
        for(j=i+1;j<n;j++){
            if(bt[i]>bt[j]){
                temp=bt[i]; bt[i]=bt[j]; bt[j]=temp;
                temp=p[i]; p[i]=p[j]; p[j]=temp;
            }
        }
    }
    wt[0]=0;
    for(i=1;i<n;i++)
        wt[i]=wt[i-1]+bt[i-1];
    for(i=0;i<n;i++)
        tat[i]=wt[i]+bt[i];
    printf("\nProcess\tBT\tWT\tTAT\n");
    for(i=0;i<n;i++)
        printf("P%d\t%d\t%d\t%d\n",p[i],bt[i],wt[i],tat[i]);
    return 0;
}
```

Q5. Write a C program to implement Preemptive SJF (Shortest Remaining Time First – SRTF) scheduling.

```
#include <stdio.h>
#include <limits.h>
int main(){
    int n, bt[20], at[20], rt[20];
    int wt[20]={0}, tat[20];
    int complete=0, t=0, min, shortest;
    printf("Enter number of processes: ");
    scanf("%d",&n);
    for(int i=0;i<n;i++){
        printf("Arrival & Burst P%d: ",i+1);
        scanf("%d %d",&at[i],&bt[i]);
        rt[i]=bt[i];
    }
    while(complete!=n){
        min=INT_MAX;
        shortest=-1;
        for(int i=0;i<n;i++){
            if(at[i]<=t && rt[i]>0 && rt[i]<min){
                min=rt[i];
                shortest=i;
            }
        }
        if(shortest==-1){ t++; continue; }
        rt[shortest]--;
        if(rt[shortest]==0){
            complete++;
            int finish=t+1;
            wt[shortest]=finish-bt[shortest]-at[shortest];
            if(wt[shortest]<0) wt[shortest]=0;
        }
        t++;
    }
    for(int i=0;i<n;i++)
        tat[i]=bt[i]+wt[i];
    printf("\nP\tWT\tTAT\n");
    for(int i=0;i<n;i++)
        printf("P%d\t%d\t%d\n",i+1,wt[i],tat[i]);
    return 0;   }
```

Q6. Write a C/Java/Python program to detect deadlock using the Wait-For Graph (WFG) algorithm.

Program must:

- **Accept number of processes and edges ($P_i \rightarrow P_j$ meaning P_i waits for P_j)**
- **Construct a directed graph**
- **Use DFS or cycle detection to identify deadlocks**

Print:

- **All cycles representing deadlocked processes**
- **How many deadlocks exist**
- **The minimal set of processes to terminate to break the cycle.**

```
#include <stdio.h>
#define MAX 20
int graph[MAX][MAX];
int visited[MAX];
int stack[MAX];
int n;
int dfs(int v)
{
    visited[v] = 1;
    stack[v] = 1;
    for(int i = 0; i < n; i++)
    {
        if(graph[v][i] == 1)
        {
            if(visited[i] == 0 && dfs(i))
                return 1;
            else if(stack[i] == 1)
                return 1;   // cycle found
        }
    }
    stack[v] = 0;
    return 0;
}
int main()
{
    int edges;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of edges: ");
```

```
    scanf("%d", &edges);
    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
            graph[i][j] = 0;
    printf("Enter edges (Pi Pj meaning Pi waits for Pj):\n");
    for(int i=0;i<edges;i++)
    {
        int u,v;
        scanf("%d %d",&u,&v);
        graph[u][v] = 1;
    }
    for(int i=0;i<n;i++)
    {
        visited[i] = 0;
        stack[i] = 0;
    }
    int deadlock = 0;
    for(int i=0;i<n;i++)
    {
        if(visited[i] == 0)
        {
            if(dfs(i))
            {
                deadlock = 1;
                break;
            }
        }
    }
    if(deadlock)
        printf("Deadlock Detected\n");
    else
        printf("No Deadlock\n");
    return 0;
}
```

Q7. Write a program to implement Banker's Algorithm to avoid deadlock.

The program should accept the Allocation matrix, Maximum matrix, and Available resources. It should calculate the Need matrix and determine whether the system is in a safe state. If the system is safe, print safe sequence.
Use the following data:

Allocation Matrix

Process	A	B	C
P1	0	1	0
P2	2	0	0
P3	3	0	2

Maximum Matrix

Process	A	B	C
P1	7	5	3
P2	3	2	2
P3	9	0	2

Available Resources

Available = (3, 3, 2)

Program should:

1. **Compute Need = Max − Allocation**
2. **Check safe state**
3. **Display safe sequence or unsafe message**

```
#include <stdio.h>
int main() {
    int n=3, m=3;
    int alloc[3][3]={{0,1,0},{2,0,0},{3,0,2}};
    int max[3][3]={{7,5,3},{3,2,2},{9,0,2}};
    int avail[3]={3,3,2};
    int need[3][3];
```

```
    int finish[3]={0};
    int safeSeq[3];
    int count=0;
    for(int i=0;i<n;i++)
        for(int j=0;j<m;j++)
            need[i][j]=max[i][j]-alloc[i][j];
    while(count<n){
        for(int i=0;i<n;i++){
            if(!finish[i]){
                int flag=0;
                for(int j=0;j<m;j++){
                    if(need[i][j]>avail[j]){
                        flag=1; break;
                    }
                }
                if(!flag){
                    safeSeq[count++]=i;
                    for(int j=0;j<m;j++)
                        avail[j]+=alloc[i][j];
                    finish[i]=1;
                }
            }
        }
    }
    if(count==n){
        printf("Safe sequence: ");
        for(int i=0;i<n;i++)
            printf("P%d ",safeSeq[i]);
    } else
        printf("Unsafe state");
    return 0;
}
```

Q8. Write a program to implement non-preemptive Priority Scheduling for the given set of processes. Calculate the waiting time and turnaround time, and display the Gantt chart.

Tabular Data for Processes

Process ID	Arrival Time	Burst Time	Priority
P1	0	6	3
P2	1	8	1
P3	2	7	4
P4	3	3	2
P5	4	4	5

```
#include <stdio.h>
struct Process{
    int id,at,bt,pr,wt,tat;
};
int main(){
    struct Process p[5]={
        {1,0,6,3},
        {2,1,8,1},
        {3,2,7,4},
        {4,3,3,2},
        {5,4,4,5}
    };
    int time=0;
    for(int i=0;i<5;i++){
        int idx=-1;
        int best=999;
        for(int j=0;j<5;j++){
            if(p[j].at<=time && p[j].bt>0 && p[j].pr<best){
                best=p[j].pr;
                idx=j;
            }
        }
        if(idx==-1){time++; i--; continue;}
        p[idx].wt=time-p[idx].at;
        time+=p[idx].bt;
        p[idx].tat=p[idx].wt+p[idx].bt;
        p[idx].bt=0;
```

```
    }
    printf("P WT TAT\n");
    for(int i=0;i<5;i++)
        printf("P%d %d %d\n",p[i].id,p[i].wt,p[i].tat);
}
```

Q9. Write a program to implement the Round Robin CPU scheduling algorithm for the following set of processes, assuming a time quantum of 4 ms.

Process	Arrival Time (ms)	Burst Time (ms)
P1	0	10
P2	1	5
P3	2	8
P4	3	6

The program should:

1. **Display the Gantt chart**
2. **Calculate the Waiting Time and Turnaround Time for each process**
3. **Compute the Average Waiting Time and Average Turnaround Time**

```
#include <stdio.h>
int main(){
    int bt[4]={10,5,8,6};
    int at[4]={0,1,2,3};
    int rt[4],wt[4]={0},tat[4];
    int q=4,t=0,done;
    for(int i=0;i<4;i++) rt[i]=bt[i];
    do{
        done=1;
        for(int i=0;i<4;i++){
            if(rt[i]>0){
                done=0;
                if(rt[i]>q){
                    t+=q;
                    rt[i]-=q;
                }else{
                    t+=rt[i];
                    wt[i]=t-bt[i]-at[i];
                    rt[i]=0;
                }
            }
```

```
        }
    }while(!done);
    for(int i=0;i<4;i++)
        tat[i]=bt[i]+wt[i];
    printf("P WT TAT\n");
    for(int i=0;i<4;i++)
        printf("P%d %d %d\n",i+1,wt[i],tat[i]);
}
```

Q10. A CPU schedules processes using the FCFS scheduling algorithm.
The following processes arrive at the system at different times:

Process	Arrival Time (ms)	Burst Time (ms)
P1	0	6
P2	2	4
P3	4	5
P4	6	3

Calculate the waiting time of each process.

Processes:
P1(0,6) P2(2,4) P3(4,5) P4(6,3)
Execution order: P1 → P2 → P3 → P4
Waiting times:
P1 = 0
P2 = 6−2 = 4
P3 = 10−4 = 6
P4 = 15−6 = 9